65. Оператор switch

На этом уроке мы рассмотрим еще один оператор управления потоком выполнения программы — оператор switch, а также то, зачем его использовать и как его делать эффективным.

Оглавление:

1. Зачем использовать оператор switch?
2. Оператор switch
3. Либо/или case
4. Либо по умолчанию
5. switch за пределами
6. switch и оператор break
7. Несколько счетчиков внутри блока switch
8. Объявление переменной и её инициализация внутри case
9. Тест

Зачем использовать оператор switch?

Есть мы в основном использовать среду наподобие операторов if-else вместе — читается и смотрится это не очень. Например:

```
enum colors
{
    CLR_RED,
    CLR_GREEN,
    CLR_BLUE,
    CLR_BLACK,
    CLR_WHITE,
    CLR_ORANGE,
    CLR_RAND
};

void printColor(colors color)
{
    if (color == CLR_RED)
        std::cout << "red";
    else if (color == CLR_GREEN)
        std::cout << "green";
    else if (color == CLR_BLUE)
        std::cout << "blue";
    else if (color == CLR_BLACK)
        std::cout << "black";
    else if (color == CLR_WHITE)
        std::cout << "white";
    else if (color == CLR_ORANGE)
        std::cout << "orange";
    else // color == CLR_RAND
        std::cout << "random";
}

int main()
{
    printColor(CLR_BLUE);
    return 0;
}
```

Использование нескольких if-блоков для проверки значения одной переменной — процесс раздражающий, но даже C++ старается сделать подобный способ более эффективный **условный оператор называется switch**. Вот вышеупомянутая программа, но уже с использованием оператора switch:

```
enum colors
{
    CLR_RED,
    CLR_GREEN,
    CLR_BLUE,
    CLR_BLACK,
    CLR_WHITE,
    CLR_ORANGE,
    CLR_RAND
};

void printColor(colors color)
{
    switch (color)
    {
        case CLR_RED:
            std::cout << "red";
            break;
        case CLR_GREEN:
            std::cout << "green";
            break;
        case CLR_BLUE:
            std::cout << "blue";
            break;
        case CLR_BLACK:
            std::cout << "black";
            break;
        case CLR_WHITE:
            std::cout << "white";
            break;
        case CLR_ORANGE:
            std::cout << "orange";
            break;
        case CLR_RAND:
            std::cout << "random";
            break;
    }
}

int main()
{
    printColor(CLR_BLUE);
    return 0;
}
```

Общая идея оператора switch проста: выражение оператора switch (например, GetChoiceOfCat() далее) проверяется значение, а каждый **case** (или **default**) проверяет это значение на соответствие. Если найс совпадает с выражением switch, то выполняется последующий код соответствующего найс-блока. Если не один найс не совпадает выражению switch, то выполняется инструкции после найс default (если он вообще указан).

Из за самой реализации, оператор switch обычно более эффективен, чем цепочка if-else. Давайте рассмотрим его более подробно.

Оператор switch

Сначала пишем **ключевое слово switch** за которым следует выражение, с которым мы хотим работать. Обычно это выражение представляет собой только одну переменную, но это может быть и почти более сложное, например, (x + y) или (x / 2). Единственное ограничение в этом выражении этом должно быть интегрального типа данных (т.е. типа char, short, int, long, long long или enum). Переменные типа с плавающей точкой или комплексные типы использоваться не могут.

После выражения switch мы объявляем блок. Внутри блока мы используем **ноЛабы** (англ. **cases**) для определения всех значений, которые мы хотим проверить на соответствие выражению. Существует два типа ноЛабов.

Либо/или case

Первый вид ноЛаба — это case (или просто **case**), который объявляется с использованием **ключевого слова case** и имеет константное выражение. Константное выражение — это то, которое генерирует константное значение, которое сможет либо быть вычислено компилятором (например, CLR_RED), либо константу (например, переменную), с которой было объявлено с ключевым словом const.

Константное выражение, находящееся после ключевого слова case, проверяется на равенство с выражением, находящимся после ключевого слова switch. Если они совпадают, то тогда выполняется код под соответствующим найсом.

Стоит отметить, что все выражения case должны производить уникальные значения. То есть мы не сможем сделать следующее:

```
switch (x)
{
    case 1:
    case 2:
    case 3:
    case 1:
    case CLR_RED:
    case CLR_GREEN:
    case CLR_BLUE:
    case CLR_RED:
}
```

Можно использовать цепь нескольких найсов для одного выражения. Следующий фрагмент использует несколько найсов для проверки, соответствует ли переменная (x) числу из ASCII-таблицы:

```
switch (x)
{
    case 'A':
    case 'B':
    case 'C':
    case 'D':
    case 'E':
    case 'F':
    case 'G':
    case 'H':
    case 'I':
    case 'J':
    case 'K':
    case 'L':
    case 'M':
    case 'N':
    case 'O':
    case 'P':
    case 'Q':
    case 'R':
    case 'S':
    case 'T':
    case 'U':
    case 'V':
    case 'W':
    case 'X':
    case 'Y':
    case 'Z':
    std::cout << x << " is an uppercase character\n";
    break;
    case '0':
    case '1':
    case '2':
    case '3':
    case '4':
    case '5':
    case '6':
    case '7':
    case '8':
    case '9':
    std::cout << x << " is a digit\n";
    break;
    case '+':
    case '-':
    case '*':
    case '/':
    std::cout << x << " is an arithmetic operator\n";
    break;
    case '(':
    case ')':
    std::cout << x << " is a parenthesis\n";
    break;
    case '[':
    case ']':
    std::cout << x << " is a bracket\n";
    break;
    case ';':
    std::cout << x << " is a semicolon\n";
    break;
    default:
    std::cout << x << " is not a special character\n";
}
```

В случае, если x равняется числу из ASCII-таблицы, то выполняется первый найс-блок после найса — (return true);

Либо по умолчанию

Второй тип найса — это **либо по умолчанию** (по-английски **default case**), который объявляется с использованием **ключевого слова default**. Код под этим найсом выполняется, если ни один из найсов не соответствует выражению switch. Найс по умолчанию необязательно. Если в switch-выражении не хватает найсов, то следующий код будет выполнен сразу после последнего блока switch.

В вышеупомянутом примере, если (x) не является числом из ASCII-таблицы, то тогда выполняется либо по умолчанию и возвращается false.

switch и fall-through

Одна из самых замороченных вещей в switch — это последовательность выполнения кода. Когда найс совпал (или выполнился default), то выполняется начиная с первого найса, который находится после соответствующего найса и продолжается до тех пор, пока не будет выполнено одно из следующих условий завершения:

- Достигнут конец блока switch.
- Выполняется оператор return.
- Выполняется оператор goto.
- Выполняется оператор break.

Обычным явлением, если не одного из этих условий завершения не будет, то выполняться будет код найса после того найса, который совпал с выражением switch. Например:

```
switch (x)
{
    case 1:
    std::cout << 1 << "\n"; // выполняется
    case 2:
    std::cout << 2 << "\n"; // выполняется тоже выполняется тоже
    case 3:
    std::cout << 3 << "\n"; // выполняется тоже выполняется тоже
    case 4:
    std::cout << 4 << "\n"; // и т.д. тоже выполняется
    case 5:
    std::cout << 5 << "\n"; // и т.д. тоже
    default:
    std::cout << 6 << "\n"; // и т.д. тоже
}
```

Результат:

```
1
2
3
4
5
6
```

А это точно не то, что нам нужно! Когда выполняется последующий за одним найсом найс в следующий, то это называется **fall-through**. Программисты почти никогда не используют fall-through, поэтому в редких случаях, когда это все-таки используется — программист оставляет комментарий, в котором сообщает, что fall-through является преднамеренным.

switch и оператор break

Оператор **break** используется для завершения **ключевого слова break**: сообщает компилятору, что мы уже сделали всё, что хотели с операционным switch (или циклом while, do while или for) и больше не намерены с ним работать. Когда компилятор встречает оператор break, выполняется код перешедит на шаг за следующим оператором switch. Обычно это объявление самых последних в блоке switch.

В вышеупомянутом примере, если x равняется числу из ASCII-таблицы, то тогда выполняется либо по умолчанию и возвращается true.

Несколько счетчиков внутри блока switch

Ещё одна странность в switch заключается в том, что вы можете использовать несколько счетчиков под каждым найсом, на пример:

```
switch (x)
{
    case 1:
    int y(5);
    int z(6);
    break;
    case 2:
    int y(7);
    break;
    default:
    int y(8);
    int z(9);
}
```

Объявление переменной и её инициализация внутри case

Вы можете объявить, но не инициализировать переменные внутри блока case:

```
switch (x)
{
    case 1:
    int y(5); // OK: объявление разрешено
    y = 6; // OK: присвоение переменной допустимо
    break;
    case 2:
    int z(7); // OK: объявление и вывод объявлено тоже, внутри до найса инициализация не нужно
    break;
    default:
    int a(8); // OK: объявлено, но не инициализировано, переменная внутри default
    break;
}
switch (x)
{
    case 1:
    int y(5); // OK: объявление разрешено
    y = 6; // OK: присвоение переменной допустимо
    break;
    case 2:
    int z(7); // OK: объявление и вывод объявлено тоже, внутри до найса инициализация не нужно
    break;
    default:
    int a(8); // OK: объявлено, но не инициализировано, переменная внутри default
    break;
}
```

Обычно внимание, что хотя переменная y была определена в первом найсе, она также используется и во втором найсе. Все переменные, определенные в switch, доступны для всех найсов, поэтому, обычно переменная в одном найсе, вы можете использовать повторно, во всех остальных найсах.

Это может показаться немного неуклюжим. Поэтому давайте рассмотрим это детально. Когда мы определяем локальную переменную, например, int (x), то переменная не создается в этой точке — это фактически создается в начале блока, в котором объявлена. Однако, это не вывод в программе до точки объявления. Сама объявление не выполняется, это просто сообщает компилятору, что переменная уже может использоваться в коде. Поэтому переменная, объявленная в одном найсе, может использоваться в другом найсе, даже если найс, объявляющий переменную, никогда не выполняется.

Однако инициализация переменной не выполняется, пока не попадаем в фактический код, который мы выполняем. Это гарантирует, что инициализация переменной произойдет выполнение, а найс, содержащий инициализацию, может никогда не выполняться!

Если в найсе нужно объявить, но не инициализировать новую переменную, то это лучше всего сделать, используя блок счетчиков внутри найса:

```
switch (x)
{
    case 1:
    int y(5);
    // OK: инициализация, также создана блок
    int z(6); // OK: инициализация, переменная видна только внутри, которая является внутри блока
    break;
    case 2:
    int y(7);
    break;
    default:
    int a(8);
    int z(9);
}
```

Обратите внимание, что хотя переменная y была определена в первом найсе, она также используется и во втором найсе. Все переменные, определенные в switch, доступны для всех найсов, поэтому, обычно переменная в одном найсе, вы можете использовать повторно, во всех остальных найсах.

Это может показаться немного неуклюжим. Поэтому давайте рассмотрим это детально. Когда мы определяем локальную переменную, например, int (x), то переменная не создается в этой точке — это фактически создается в начале блока, в котором объявлена. Однако, это не вывод в программе до точки объявления. Сама объявление не выполняется, это просто сообщает компилятору, что переменная уже может использоваться в коде. Поэтому переменная, объявленная в одном найсе, может использоваться в другом найсе, даже если найс, объявляющий переменную, никогда не выполняется.

Однако инициализация переменной не выполняется, пока не попадаем в фактический код, который мы выполняем. Это гарантирует, что инициализация переменной произойдет выполнение, а найс, содержащий инициализацию, может никогда не выполняться!

Если в найсе нужно объявить, но не инициализировать новую переменную, то это лучше всего сделать, используя блок счетчиков внутри найса:

```
switch (x)
{
    case 1:
    int y(5);
    // OK: инициализация, также создана блок
    int z(6); // OK: инициализация, переменная видна только внутри, которая является внутри блока
    break;
    case 2:
    int y(7);
    break;
    default:
    int a(8);
    int z(9);
}
```

Правило: Если нужно инициализировать или объявить переменную внутри найса — используйте блок счетчиков.

Тест

Задание №1

Напишите функцию calculate(), которая принимает две переменные типа int и одну переменную типа char, которая, в свою очередь, представляет одну из следующих математических операций: +, -, *, / или %. Используйте switch для определения какой из операций выполнить, и возвращайте результат. Если пользователь ввел неизвестный оператор, то функция должна выводить ошибку. С оператором должна выполняться целочисленная деление.

Задание №2

Определите перечисление (или enum enum) animal, которое содержит следующие элементы: pig, chicken, goat, cat, dog и cattyo. Напишите функцию getNumberOfLegs(), которая принимает параметр animal, и использует switch для возврата того количества в качестве целого. Напишите еще одну функцию — printNumberOfLegs(), которая использует switch для вывода следующего: "pig has 4 legs\n" (или "chicken has 2 legs\n" и т.д.). Наконец, что для каждого типа animal вывод, который выводит сообщение об ошибке. Выражение printNumberOfLegs(4) должно, используя в качестве параметров cat и chicken.

Пример результата выполнения вашей программы:

```
A cat has 4 legs.
A chicken has 2 legs.
```

Ответ 1

```
#include <iostream>

int calculate(int x, int y, char op)
{
    switch (op)
    {
        case '+':
        return x + y;
        case '-':
        return x - y;
        case '*':
        return x * y;
        case '/':
        return x / y;
        case '%':
        return x % y;
        default:
        std::cout << "calculate(): Unhandled case\n";
        return 0;
    }
}

int main()
{
    std::cout << "Enter an integer: ";
    int x;
    std::cin >> x;

    std::cout << "Enter another integer: ";
    int y;
    std::cin >> y;

    std::cout << "Enter a mathematical operator (+, -, *, /, or %): ";
    char op;
    std::cin >> op;

    std::cout << x << " " << op << " " << y << " is " << calculate(x, y, op) << "\n";

    return 0;
}
```

Ответ №2

```
#include <iostream>

enum animal
{
    ANIMAL_PIG,
    ANIMAL_CHICKEN,
    ANIMAL_GOAT,
    ANIMAL_CAT,
    ANIMAL_DOG,
    ANIMAL_CATTYO
};

std::string getAnimalName(enum animal)
{
    switch (animal)
    {
        case ANIMAL_CHICKEN:
        return "chicken";
        case ANIMAL_CAT:
        return "cat";
        case ANIMAL_PIG:
        return "pig";
        case ANIMAL_GOAT:
        return "goat";
        case ANIMAL_DOG:
        return "dog";
        default:
        return "getAnimalName(): Unhandled enumerator";
    }
}

void printNumberOfLegs(Animal animal)
{
    std::cout << "A " << getAnimalName(animal) << " has ";

    switch (animal)
    {
        case ANIMAL_CHICKEN:
        std::cout << "2";
        break;
        case ANIMAL_CATTYO:
        std::cout << "2";
        break;
        case ANIMAL_PIG:
        std::cout << "4";
        break;
        case ANIMAL_GOAT:
        std::cout << "4";
        break;
        case ANIMAL_CAT:
        std:: cout << "4";
        break;
        case ANIMAL_DOG:
        std::cout << "4";
        break;
        default:
        std::cout << "printNumberOfLegs(): Unhandled enumerator";
        break;
    }

    std::cout << " legs.\n";
}

int main()
{
    printNumberOfLegs(ANIMAL_CAT);
    printNumberOfLegs(ANIMAL_CHICKEN);

    return 0;
}
```